

LLM Chatbot System Report

Technical Report: LLM-Powered Chatbot System — Reference Implementation

Source file: LLM_Chatbot_System (3).ipynb **Platform:** Google Colab (CPU or T4 GPU) **Report date:** August 14, 2026

1. Purpose and Positioning

This notebook is a **runnable reference implementation of a chatbot system described in an accompanying “System Requirement & Implementation” (SRII) report**. Rather than being a standalone demo, it is explicitly designed as a working companion to formal system-design documentation: each module in the notebook is annotated with the specific section of the SRII report it implements (architecture diagram, ER diagram, module descriptions, process flowchart, security requirements, technology stack, and UI screenshots).

This makes it functionally different from a typical exploratory notebook — it’s an **executable specification**, built to demonstrate that a documented system design actually works end-to-end, with every architectural box in the design mapped to a real class.

Like the “Recall” notebook, it runs with **no API key** — a small local Hugging Face model (Qwen2.5-0.5B-Instruct) stands in for a paid LLM API, keeping the whole system self-contained on Colab’s free tier.

2. Architecture-to-Code Mapping

The notebook’s structure directly mirrors a layered chatbot system design:

SRII Section	System component	Notebook implementation
4.1 System Architecture	Orchestrator	Chatbot class
4.5 Database Design (ER diagram)	USER → SESSION → CHAT_MESSAGE → FEEDBACK, KNOWLEDGE_DOC	SessionStore + KnowledgeBase
4.6 Module Description	Auth, Session Manager, Retrieval, Prompt Builder, LLM Client, Post-Processor, Logger	One class per module
4.7 Process Flow	Message-handling flowchart	Chatbot.chat()
4.9 Security	Hashed passwords, server-side key, stateless orchestrator, safety filter	AuthModule , ResponsePostProcessor
5.3 / 5.7 UI (chat window, feedback panel)	Interactive chat + star rating	ipywidgets UI
5.6 Admin analytics dashboard	Latency / retrieval-usage charts	AnalyticsLogger + matplotlib

This 1:1 traceability between design document and code is the notebook’s defining feature — it reads like a proof that the SRII’s architecture is implementable, not just diagrammed.

3. Component-by-Component Breakdown

3.1 Dependencies (Cell 2)

```
transformers, accelerate, scikit-learn, ipywidgets, pandas, matplotlib
```

Notably lighter than a production RAG stack — no vector database, no external embedding service. `scikit-learn` is doing double duty as the retrieval engine (see 3.3).

3.2 Authentication & Session Manager (Cells 5-7) — FR-1, Section 4.9

```
class AuthModule:
    def _hash_password(password, salt="srii_salt"):
        return hashlib.sha256((salt + password).encode()).hexdigest()
```

- Implements register / login / token-based `authenticate()` against an in-memory `_users` and `_tokens` dict.
- Passwords are **hashed with salted SHA-256**, never stored in plaintext — mirrors the security requirement in 4.9, though note SHA-256 (even salted) is not a recommended password-hashing algorithm for production (see §5 below).
- Session tokens are plain UUIDs with no expiry — functionally similar to a JWT in the design doc, but without JWT's built-in expiry/claims, which the report's 4.9 section implies exists ("short-lived session token").

`SessionStore` (Cell 7) models the ER diagram directly: - `ChatMessage` dataclass = the `CHAT_MESSAGE` entity. - `sessions: Dict[session_id, {user_id, created_at}] = SESSION`. - `add_feedback()` mutates a message's `feedback` field = `FEEDBACK`, modeled as an attribute of `CHAT_MESSAGE` rather than a separate table — a simplification worth flagging if the ER diagram in the report shows `FEEDBACK` as its own entity with its own primary key. - `get_history(session_id, max_turns=10)` implements **context-window management** by returning only the last `max_turns * 2` messages (user+assistant pairs) — directly addresses the "context management" challenge called out in the report's Section 2.

3.3 Retrieval Module — Knowledge Base (Cell 9) — Section 4.1/4.6, RAG

```
class KnowledgeBase:
    def retrieve(self, query, top_k=2, min_score=0.05):
        ...
        return [doc for score, doc in ranked[:top_k] if score >= min_score]
```

- Uses **TF-IDF + cosine similarity** (scikit-learn) rather than a true embedding-based vector store. The notebook is explicit about this being a stand-in for a production vector DB (pgvector/Pinecone/Chroma) — a reasonable simplification to keep the notebook dependency-light, but it means retrieval is **lexical, not semantic**: queries that paraphrase the knowledge base without sharing keywords will not retrieve relevant documents.
- The `min_score` threshold implements the **conditional retrieval decision point** from the process flowchart (Fig 4.7) — retrieval only "fires" when similarity clears a bar, rather than always injecting KB content. This is a nice detail that matches good RAG practice (avoid injecting irrelevant context) and is more disciplined than some naive RAG implementations.
- Sample knowledge base is 4 short domain FAQ-style sentences (support hours, refunds, pricing, password reset) — enough to demonstrate the mechanism, not a stress test of retrieval quality at scale.

3.4 Prompt Builder (Cell 11) — Conversation Orchestrator

```
class PromptBuilder:
    @staticmethod
    def build_messages(history, retrieved_context):
        ...
```

- Injects retrieved KB snippets as a synthetic **user → assistant acknowledgment pair**

("[Reference context...]" / "Understood, I will use this context if relevant.") prepended to the real conversation history, rather than using a `system`-role message for context. This is a workable pattern for models whose chat templates handle multiple system-like turns awkwardly, but it does consume two message slots per turn and could be replaced with a single content-augmented system message for efficiency.

- History and retrieved context are merged into a flat message list, later combined with `SYSTEM_PROMPT` inside `LLMClient.generate()`.

3.5 LLM Inference Client (Cell 13) — Section 4.1/4.8

```
DEFAULT_MODEL = "Qwen/Qwen2.5-0.5B-Instruct"
```

- Uses an even smaller model than the Recall notebook (0.5B vs. 1.5B parameters) — appropriate here since the notebook's focus is system architecture and data flow, not generation quality, and a smaller model means faster CPU-only fallback (`device = "cuda" if torch.cuda.is_available() else "cpu"`).
- `LLMClient` is deliberately designed as a **swappable interface**: "same interface as an API-backed client would have... without touching the orchestrator." This directly demonstrates the layered/decoupled design principle from Section 4.8 — a genuinely good architectural point to highlight in a viva, since it shows the vendor/model choice is isolated behind one class boundary.
- `do_sample=temperature > 0` is a nice touch — it makes the sampling behavior properly deterministic when `temperature=0`, rather than always sampling.

3.6 Response Post-Processor (Cell 15) — Section 2 & 4.9, Safety Filtering

```
BLOCKED_PATTERNS = [  
    r"\bhow to (make|build) (a )?(bomb|weapon)\b",  
    r"\bself[- ]harm\b",  
]
```

- A minimal regex-based content-safety gate applied to **both** the user message and the raw model output before returning a response — checking the input as well as the output is a sound design choice (catches unsafe requests even if the small model doesn't clearly refuse them).
- Explicitly flagged in the notebook as a stand-in for a real moderation API — appropriately scoped as a placeholder rather than a production safety system, and the report should probably state this limitation explicitly if it's presented as satisfying the 4.9 security requirement.

3.7 Logging, Analytics & Feedback (Cell 17) — Section 4.6/FR-7

```
class AnalyticsLogger:  
    def log_turn(...): ...  
    def summary(self): ...
```

- Logs every turn (session id, message id, both messages, whether retrieval fired, latency, timestamp) into a list of dicts, exposed as a pandas DataFrame.
- `summary()` computes turn count, average latency, % of turns that used retrieval, and average user rating — a clean, minimal analytics layer that maps well onto an admin dashboard screenshot (Section 5.6).
- Feedback is recorded in **two places** (`SessionStore.add_feedback` and `AnalyticsLogger.add_feedback`) — functionally fine for a demo but a duplication that a real implementation would likely resolve by having analytics read from the session store rather than maintaining a parallel copy.

3.8 Orchestrator — the `Chatbot` Class (Cell 19) — Section 4.1/4.7

```
def chat(self, token, session_id, user_message) -> Dict:  
    # 1. Authenticate  
    # 2. Persist user message  
    # 3. Retrieval (conditional)  
    # 4. Build prompt  
    # 5. LLM inference
```

```
# 6. Post-process
# 7. Persist assistant message
# 8. Log
```

This is the clearest artifact of “process flow → code” translation in the notebook. Each numbered step in the method body corresponds one-to-one with a decision/action box implied by a typical chatbot flowchart, including the **conditional retrieval branch** (step 3) and **safety post-processing** (step 6) as separate, explicit stages rather than being folded into generation. This is good structure for a viva walkthrough — you can point at the flowchart and the code side-by-side and show they match step for step.

3.9 Interactive Chat UI (Cell 23) — Section 5.3/5.7

- Built with `ipywidgets`: a scrollable output log, text input, send button, and a **per-response five-star feedback row** with an optional comment box, dynamically rendered after each assistant turn.
- Runs natively inside Colab without extra servers (unlike the Recall notebook’s Gradio approach), which is a reasonable choice here since this notebook’s emphasis is system correctness/traceability rather than a shareable public demo.

3.10 Analytics Dashboard (Cells 25-26) — Section 5.6

- Renders the `AnalyticsLogger` summary as a printed dict plus a DataFrame view, then two matplotlib charts: latency per turn (line) and retrieval usage (bar). Directly visualizes the two most operationally relevant metrics — response speed and how often RAG is contributing — in a way that would map cleanly onto an admin-facing dashboard screenshot in the report.

3.11 “Extending This Notebook” (Cell 27)

The notebook closes with a well-organized set of upgrade paths, explicitly tied back to the report’s Future Scope (6.1): swapping TF-IDF for a real vector DB, adding persistence (SQLite/Postgres), multi-user login UI, a bigger local model or hosted API, streaming responses, and voice/multilingual support. This is a strong practice for a viva-facing notebook — it preempts the “what would you do differently at scale” question.

4. Strengths

- **Traceability.** Nearly every class and comment cites the specific report section it implements — this is unusually disciplined for a notebook and makes it easy to defend in a viva: an examiner can ask “show me where FR-1 is implemented” and you can point directly at `AuthModule`.
- **Clean separation of concerns matching the layered architecture.** Auth, sessions, retrieval, prompting, inference, safety, and logging are all separate classes composed only inside `Chatbot.chat()` — this directly demonstrates the “vendor/model is decoupled” claim in 4.8 rather than just asserting it.
- **Conditional retrieval, not blind RAG.** The `min_score` gate means the system only augments prompts with KB content when it’s actually relevant, which is more defensible in a design review than always concatenating retrieved text.
- **End-to-end observability.** Every turn is logged with latency and retrieval usage, and feedback closes the loop back to specific messages — this gives you real data to discuss (e.g., “average latency was X ms, Y% of turns used retrieval”) rather than only a qualitative demo.
- **Runs with zero setup cost.** No API key, no external services, no signup — consistent with your Recall notebook’s design philosophy, and appropriate for a reproducible academic submission.

5. Notable Gaps / Risks (worth addressing or at least acknowledging in the report)

1. **Password hashing.** Salted SHA-256 is fast by design, which makes it weak against offline brute-

force/rainbow-table attacks compared to a purpose-built KDF (bcrypt, scrypt, or Argon2). Worth a one-line caveat in the security section: “for demonstration purposes; production would use bcrypt/Argon2.”

2. **Session tokens never expire.** The report’s 4.9 language (“short-lived session token,” JWT-based) implies expiry, but `AuthModule._tokens` has no TTL or invalidation logic. If an examiner reads both documents closely, this is a mismatch worth either fixing (add an expiry timestamp check in `authenticate()`) or noting as a known simplification.
3. **Lexical (TF-IDF) retrieval, not semantic.** This is already flagged in the notebook itself, but worth restating clearly in the report: retrieval quality will degrade on paraphrased or vocabulary-mismatched queries, which is a meaningful limitation if the report’s RAG section implies embedding-based semantic search.
4. **In-memory-only persistence.** `SessionStore`, `KnowledgeBase`, and `AnalyticsLogger` all reset on kernel restart — fine for a Colab demo, explicitly flagged as a “swap for SQLite/Postgres” future step, but worth being upfront that no data survives a runtime disconnect (relevant if your viva panel asks about durability).
5. **Small model quality ceiling.** At 0.5B parameters, expect noticeably weaker instruction-following and factual grounding than the report’s design intent might suggest if it was written around a GPT-4-class model — similar caveat to the Recall notebook, but even more pronounced here given the smaller size.
6. **Regex-based safety filter has limited coverage.** Two hardcoded patterns will miss most unsafe content; this is explicitly acknowledged as a placeholder, but it’s worth stating plainly in the report that this does not constitute a real moderation system.
7. **Duplicate feedback storage.** `SessionStore.add_feedback` and `AnalyticsLogger.add_feedback` maintain separate copies of the same rating — a minor inconsistency risk if one is updated and not the other.

6. Suggested Talking Points for a Viva

- Be ready to walk from a flowchart box → the corresponding line(s) in `Chatbot.chat()` — this notebook is built for exactly that kind of live mapping.
 - Have the caveats in §5 ready as “known simplifications, here’s what production would need” answers — examiners often probe exactly these points (security, persistence, retrieval quality), and having a ready, precise answer is stronger than being caught off guard.
 - The `LLMClient` abstraction is a good example to cite if asked “how would you swap in GPT-4/Claude” — you can show the interface boundary explicitly rather than describing it abstractly.
 - The analytics dashboard gives you real numbers (latency, retrieval-usage %) to quote rather than only describing the system qualitatively.
-

7. Summary

This notebook is a disciplined, well-annotated executable companion to a formal chatbot system design document, with strong 1:1 traceability between architecture sections and code modules. Its main value is demonstrating that a layered, security- and analytics-aware chatbot architecture is implementable end-to-end on free infrastructure — the main things to be upfront about if challenged are the simplified password hashing, non-expiring tokens, lexical-only retrieval, in-memory persistence, and the placeholder nature of the safety filter, all of which are reasonable simplifications for a reference implementation as long as they’re acknowledged rather than presented as production-ready.